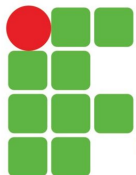




Seminário de Estrutura de Dados Lineares: TreeSort

Alunos: Franklin e Joana



INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
RIO GRANDE DO NORTE

Introdução



O Tree Sort (ou ordenação por árvore) é um algoritmo de ordenação que funciona inserindo todos os elementos de uma lista em uma Árvore Binária de Busca (BST). Como essa árvore organiza os dados colocando valores menores à esquerda e maiores à direita, basta percorrê-la na ordem correta para obter os dados ordenados.

Como funciona o algoritmo?



1° Passo - construção da Árvore: O algoritmo lê cada número da sua lista desordenada e o insere na árvore. Se o número for menor que o nó atual, ele vai para o ramo esquerdo; se for maior, para o ramo direito.

2° Passo - travessia em Ordem (In-Order Traversal): Após inserir todos os itens, o algoritmo percorre a árvore visitando o ramo esquerdo, depois a raiz e, por fim, o ramo direito. Isso garante que os elementos sejam extraídos do menor para o maior.

Exemplo Numérico Demonstrativo do Funcionamento



1º Passo

Lista desordenada: 5 8 2 4 9 1 6

Nó raiz: 5

Para inserir o 8: se ele é maior que o nó raiz, ficará a direita. Se é menor, ficará a esquerda.

5 | 8

2 | 5 | 8

2, 4 | 5 | 8

2, 4 | 5 | 8, 9

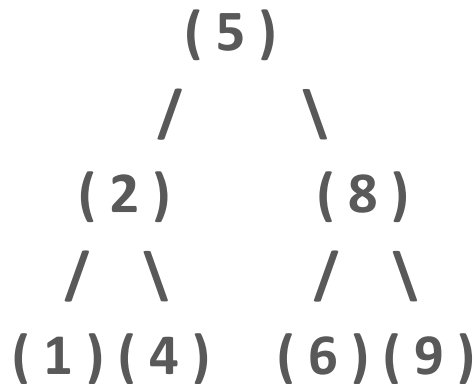
1, 2, 4 | 5 | 8, 9

1, 2, 4 | 5 | 6, 8, 9

Exemplo Numérico Demonstrativo do Funcionamento



2º Passo



Percurso em ordem: 1, 2, 4, 5, 6, 8, 9.

Código exemplo



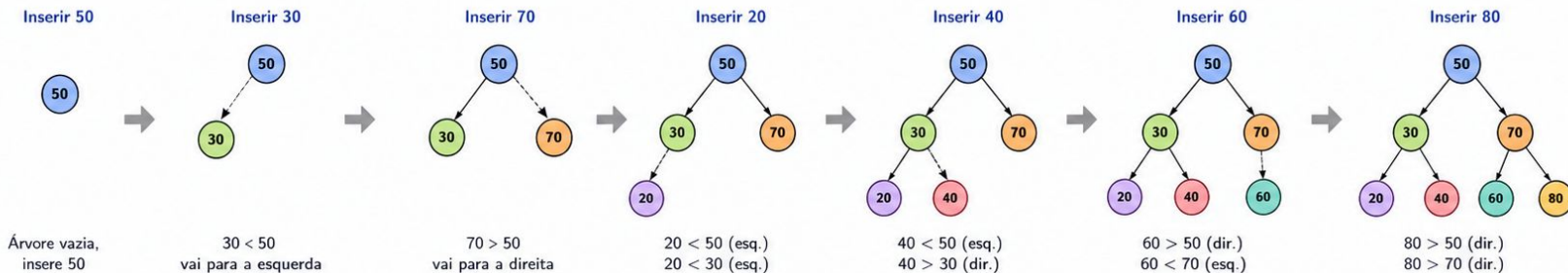
```
public class TreeSort {  
    private No raiz;  
  
    // Inserção  
    public void inserir(int valor) {  
        raiz = inserirRec(raiz, valor);  
    }  
    private No inserirRec(No atual, int valor) {  
        if (atual == null)  
            return new No(valor);  
        if (valor < atual.valor)  
            atual.esquerda = inserirRec(atual.esquerda, valor);  
        else  
            atual.direita = inserirRec(atual.direita, valor);  
        return atual;  
    }  
}
```

```
// Percurso em ordem  
public void emOrdem(No no, ArrayList<Integer> lista) {  
    if (no != null) {  
        emOrdem(no.esquerda, lista);  
        lista.add(no.valor);  
        emOrdem(no.direita, lista);  
    }  
}  
  
// Ordenação  
public ArrayList<Integer> ordenar() {  
    ArrayList<Integer> lista = new ArrayList<>();  
    emOrdem(raiz, lista);  
    return lista;  
}  
}
```

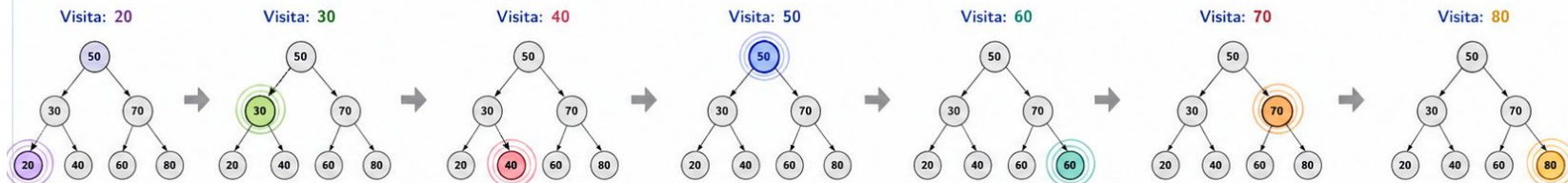
Código exemplo



1) Inserir todos os elementos na Árvore Binária de Busca



2) Percorrer a árvore em ordem (esquerda → raiz → direita)



3) Resultado: elementos em ordem crescente

20 → 30 → 40 → 50 → 60 → 70 → 80

Saída:
[20, 30, 40, 50, 60, 70, 80]

● Raiz
● Nível 1
● Nível 2

Análise do Pior Caso (Big O)



- **Tempo Médio:** $O(n \log n)$ – Ideal para a maioria dos casos em que a árvore é construída de forma balanceada.
- **Pior Caso:** $O(n^2)$ – Ocorre se a lista original já estiver ordenada ou quase ordenada, transformando a árvore em uma estrutura linear semelhante a uma lista encadeada.

Para garantir o melhor desempenho, é comum usar árvores auto-balanceáveis (como AVL ou Rubro-Negra), o que garante que a complexidade continue sendo $O(n \log n)$ até mesmo no pior caso.



Obrigada!